

◆ **A race of two compilers:**
◆ **Graal JIT vs. C2 JIT.**
Which one offers better runtime performance?

by Ionuț Balosin

IonutBalosin.com

 [@IonutBalosin](https://twitter.com/IonutBalosin)

Software Architect



**Raiffeisen Bank
International**

Trainer

< for more details visit IonutBalosin.com/training >

JAVA PERFORMANCE TUNING

SOFTWARE ARCHITECTURE

Writer

IonutBalosin.com

InfoQ

Speaker



Today's Content

- 01 **Intro**
- 02 **Scalar Replacement**
- 03 **Virtual Calls**
- 04 **Vectorization & Loop Optimizations**
- 05 **Summary & Takeaways**

Intro

01

Source: [IonutBalosin.com/2019/04/jvm-jit-compilers-benchmarks-report-19-04]



JVM JIT Compilers Benchmarks Report 19.04

Configuration

CPU	Intel i7-8550U Kaby Lake R
MEMORY	16GB DDR4 2400 MHz
OS / Kernel	Ubuntu 18.10 / 4.18.0-25-generic
Java Version	13 Linux-x64
JVM	OpenJDK
JMH v1.21	5x10s warm-up iterations, 5x10s measurement iterations, 3 JVM forks, single threaded

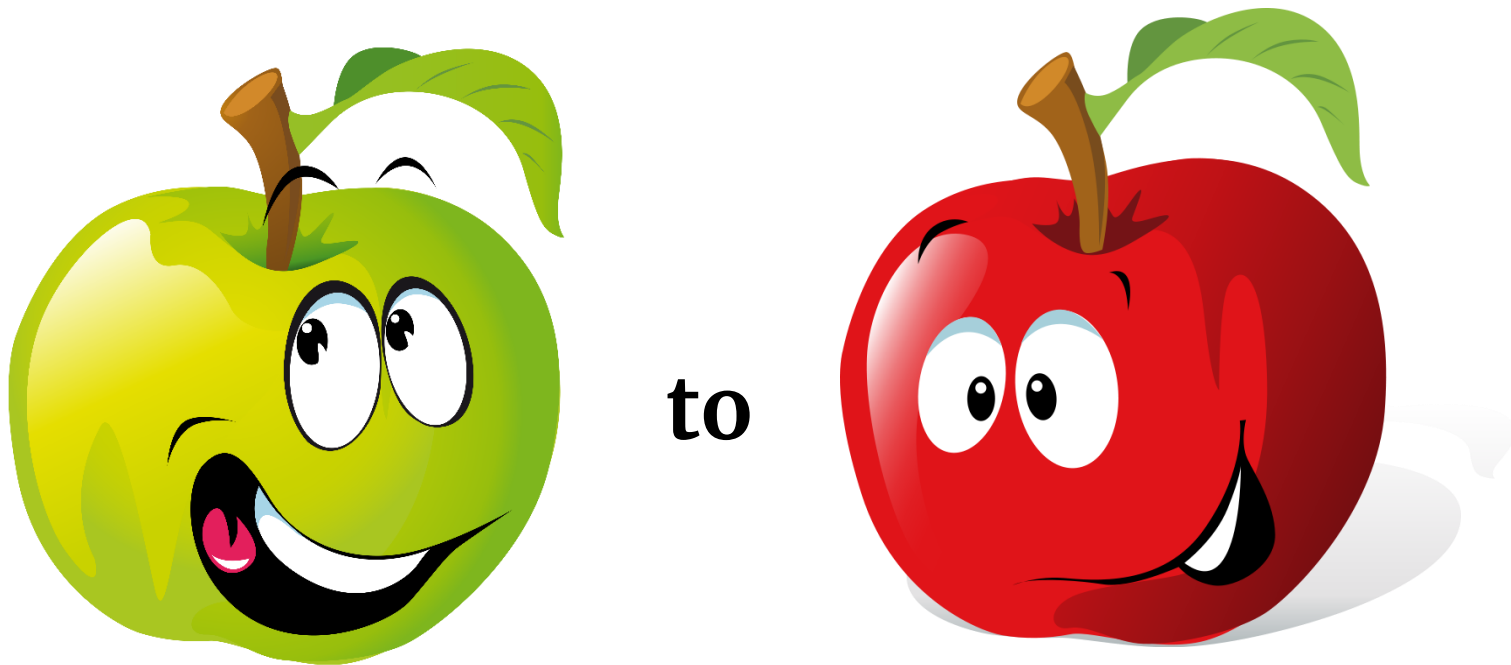
Note: C2 JIT usually needs less warm-up iterations, being a native Compiler.



This talk is NOT about GraalVM™ EE !

Current optimizations might differ for other JVMs!

Fair comparison?



Fair comparison?

Well ... it depends from the perspective:

Application/end user: performance and (licensing) costs are the leading factors

Compiler/internals (e.g. implementation, maturity): very different



Graal JIT

- ✓ Java implementation
- ✓ still early stage (recently become production ready; e.g. v19.2.0.1)
- ✓ targets JVM-based languages (e.g. Java, Scala, Kotlin), dynamic languages (e.g. JavaScript, Ruby, R), native languages (e.g. C/C++, Rust)
- ✓ modular design (JVMCI plug-in arch)
- ✓ Ahead Of Time flavor (our of scope for today's presentation)

Note: non of these compilers are in maintenance.



Graal JIT

- ✓ Java implementation
- ✓ still early stage (recently become production ready; e.g. v19.2.0.1)
- ✓ targets JVM-based languages (e.g. Java, Scala, Kotlin), dynamic languages (e.g. JavaScript, Ruby, R), native languages (e.g. C/C++, Rust)
- ✓ modular design (JVMCI plug-in arch)
- ✓ Ahead Of Time flavor (our of scope for today's presentation)

C2 JIT



- ✓ C++ implementation
- ✓ mature / production grade runtime
- ✓ written mainly to optimize Java source code
- ✓ is essentially a local maximum of performance and is reaching the end of its design lifetime
- ✓ latest improvements were more focused on intrinsics and vectorization, rather than adding new optimization heuristics

Note: non of these compilers are in maintenance.

Nevertheless, nowadays the baseline for Graal JIT still remains C2 JIT!

The target is to leverage its performance to be at least on par with C2 JIT especially on real world applications / production code.

Few main distinguishing factors for **Graal JIT**:

Partial Escape Analysis

Improved inlining

Guard optimizations for efficient
handling of speculative code

Main Graal Publications

1. Partial Escape Analysis (PEA) determines the escapability of objects on individual branches and reduces object allocations even if the objects escapes on some paths.

PEA works on compiler's intermediate representation not on bytecode. It is effective in conjunction with other parts of the compiler, such as *inlining*, *global value numbering*, and *constant folding*.

Partial Escape Analysis and Scalar Replacement for Java

Lukas Stadler
Johannes Kepler University
Linz, Austria
lukas.stadler@jku.at

Thomas Würthinger
Oracle Labs
thomas.wuerthinger
@oracle.com

Hanspeter Mössenböck
Johannes Kepler University
Linz, Austria
moessenboeck@ssw.jku.at

2. Improved inlining strategy explores the call tree and performs **late inlining** as default instead of inlining during bytecode parsing.

Graal JIT devise three novel heuristics: *callsite clustering*, *adaptive decision thresholds*, and *deep inlining trials*.

An Optimization-Driven Incremental Inline Substitution Algorithm for Just-in-Time Compilers

Aleksandar Prokopec

Oracle Labs, Switzerland

aleksandar.prokopec@gmail.com

Gilles Duboscq

Oracle Labs, Switzerland

gilles.m.duboscq@oracle.com

David Leopoldseder

JKU Linz, Austria

david.leopoldseder@jku.at

Thomas Würthinger

Oracle Labs, Switzerland

thomas.wuerthinger@oracle.com

3. Guard optimizations for efficient handling of speculative code has its biggest impact on the JavaScript, Ruby, and R (i.e. dynamic languages) implementations built on Graal's multi-language framework Truffle.

An Intermediate Representation for Speculative Optimizations in a Dynamic Compiler

Gilles Duboscq* Thomas Würthinger[†] Lukas Stadler* Christian Wimmer[†] Doug Simon[†]
Hanspeter Mössenböck*

*Institute for System Software, Johannes Kepler University Linz, Austria [†]Oracle Labs
{duboscq, stadler, moessenboeck}@ssw.jku.at
{thomas.wuerthinger, christian.wimmer, doug.simon}@oracle.com

Scalar Replacement

02

Escape Analysis (EA) analyses the scope of a new object and decides whether it might be allocated or not on the heap.

EA is not an optimization but rather an **analysis phase for the optimizer**. The result of this analysis allows compiler to perform optimizations like *object allocations*, *synchronization primitives* and *field accesses*.

Object Scopes

- ✓ **NoEscape** (local to current method/thread, not visible outside)
- ✓ **ArgEscape** (passed as an argument but not visible by other threads)
- ✓ **GlobalEscape** (visible outside of current method/thread)

For **NoEscape** objects compiler can remap the accesses to the object fields to accesses to synthetic local operands: **Scalar Replacement** optimization.

```
@Param({"3"})    private int param;

@Benchmark
public int no_escape() {
    LightWrapper w = new LightWrapper(param);
    return w.f1 + w.f2;
}

public static class LightWrapper {
    public int f1;    // param
    public int f2;    // param << 1
    //...
}
```

```
@Param({"3"})    private int param;
```

```
@Benchmark
```

```
public int no_escape() {  
    LightWrapper w = new LightWrapper(param);  
    return w.f1 + w.f2;  
}
```

equivalent to

```
public static class LightWrapper {  
    public int f1;    // param  
    public int f2;    // param << 1  
    //...  
}
```

```
@Param({"3"})    private int param;
@Param({"128"})  private int size;

@Benchmark
public int no_escape_object_array() {
    HeavyWrapper w = new HeavyWrapper(param, size);
    return w.f1 + w.f2 + w.z.length;
}

public static class HeavyWrapper {
    public int f1;    // param
    public int f2;    // param << 1
    public byte z[]; // new byte[size]
    // ...
}
```



```
@Param({"3"})    private int param;
```

```
@Param({"128"}) private int size;
```

```
@Benchmark
```

```
public int no_escape_object_array() {
```

```
    HeavyWrapper w = new HeavyWrapper(param, size);
```

```
    return w.f1 + w.f2 + w.z.length;
```

```
}
```

```
public static class HeavyWrapper {
```

```
    public int f1;    // param
```

```
    public int f2;    // param << 1
```

```
    public byte z[]; // new byte[size]
```

```
    // ...
```

```
}
```

equivalent to

```
return f1 + f2 + length;
```

```
@Param(value = {"false"}) private boolean objectEscapes;
```

```
@Benchmark
```

```
public void branch_escape_object(Blackhole bh) {  
    HeavyWrapper wrapper = new HeavyWrapper(param, size);  
    HeavyWrapper result;  
  
    if (objectEscapes) // false  
        result = wrapper;  
    else  
        result = globalCachedWrapper;  
  
    bh.consume(result);  
}
```

```
@Param(value = {"false"}) private boolean objectEscapes;
```

```
@Benchmark
```

```
public void branch_escape_object(Blackhole bh) {  
    HeavyWrapper wrapper = new HeavyWrapper(param, size);  
    HeavyWrapper result;  
  
    if (objectEscapes) // false  
        result = wrapper; ← branch is never taken,  
    else                                     wrapper never escapes,  
        result = globalCachedWrapper;    its scope is NoEscape  
  
    bh.consume(result);  
}
```

```
@Param(value = {"false"}) private boolean objectEscapes;
```

```
@Benchmark
```

```
public void branch_escape_object(Blackhole bh) {  
    HeavyWrapper wrapper = new HeavyWrapper(param, size);  
    HeavyWrapper result;  
  
    if (objectEscapes) // false  
        result = wrapper; ← branch is never taken,  
    else                                     wrapper never escapes,  
        result = globalCachedWrapper;      its scope is NoEscape  
  
    bh.consume(result);  
}
```



Based on **Partial Escape Analysis**, the **wrapper** object allocation is eliminated.

@Benchmark

```
public boolean arg_escape_object() {  
    HeavyWrapper wrapper1 = new HeavyWrapper(param, size);  
    HeavyWrapper wrapper2 = new HeavyWrapper(param, size);  
    boolean match = false;  
  
    if (wrapper1.equals(wrapper2))  
        match = true;  
  
    return match;  
}
```

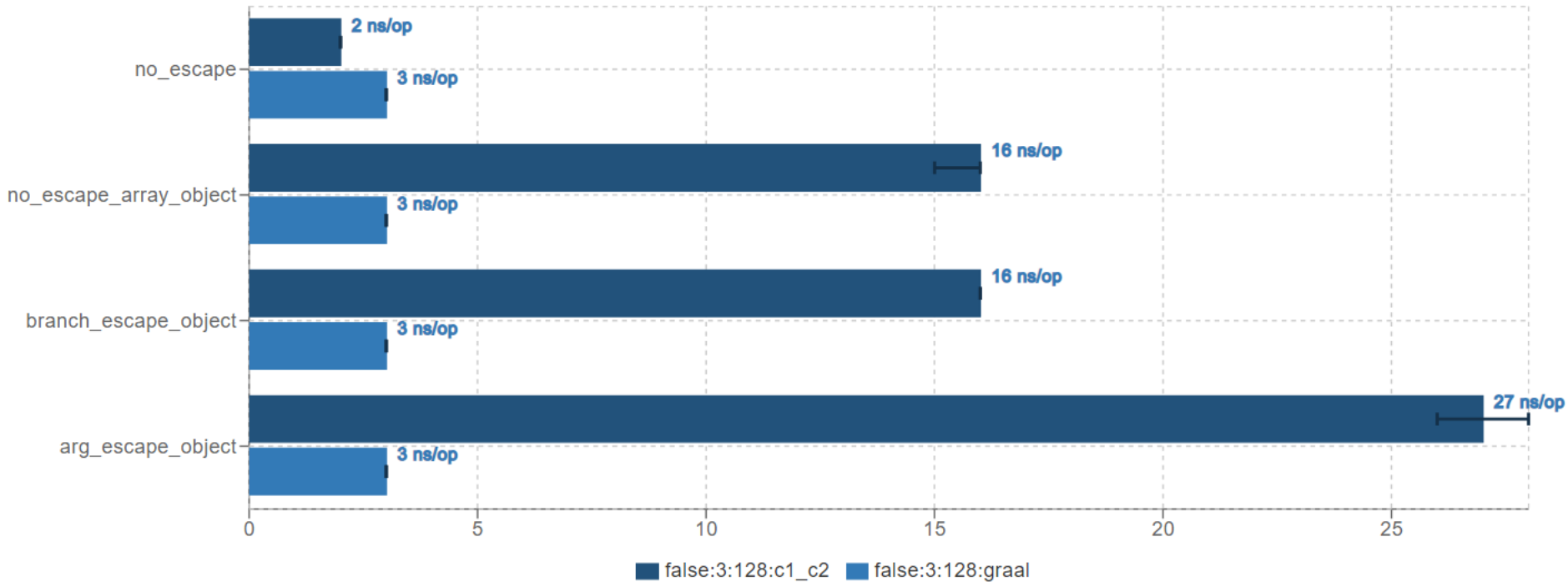
@Benchmark

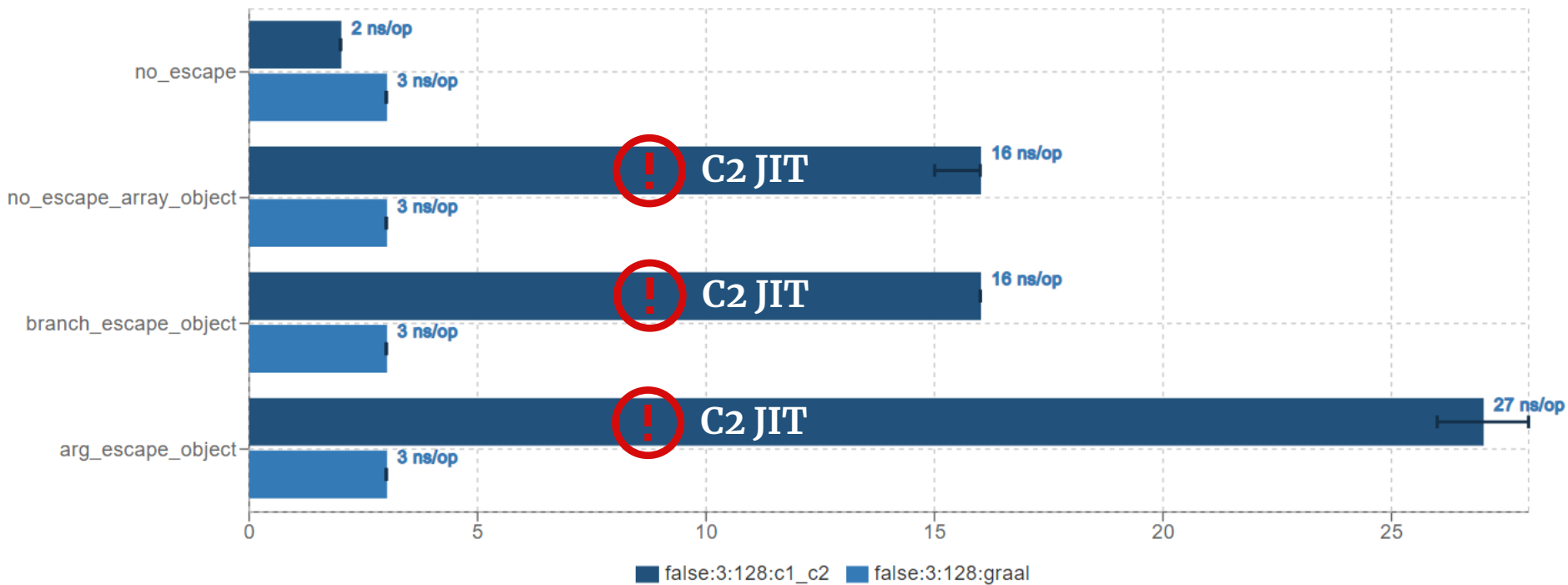
```
public boolean arg_escape_object() {  
    HeavyWrapper wrapper1 = new HeavyWrapper(param, size);  
    HeavyWrapper wrapper2 = new HeavyWrapper(param, size);  
    boolean match = false;  
  
    if (wrapper1.equals(wrapper2))  
        match = true;  
  
    return match;  
}
```

wrapper1 scope is NoEscape

wrapper2 scope is:

- NoEscape, if inlining of equals() succeeds
- ArgEscape, if inlining fails or disabled





Conclusions

Graal JIT was able to get rid of heap allocations offering a constant response time, around 3 ns/op

Conclusions

Graal JIT was able to get rid of heap allocations offering a constant response time, around 3 ns/op

C2 JIT struggles to get rid of the heap allocations in case:

- there is a condition which does not make obvious at compile time if the object escapes
- or if the object scope becomes local (i.e. NoEscape) after inlining

Conclusions

Graal JIT was able to get rid of heap allocations offering a constant response time, around 3 ns/op

C2 JIT struggles to get rid of the heap allocations in case:

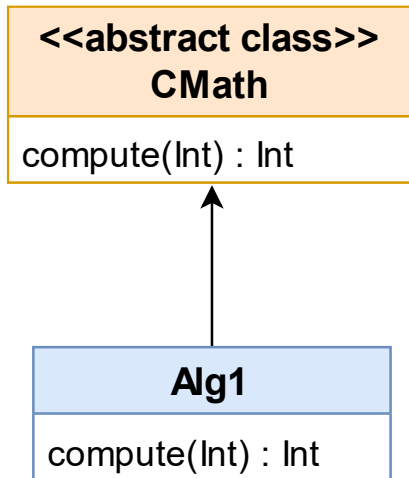
- there is a condition which does not make obvious at compile time if the object escapes
- or if the object scope becomes local (i.e. NoEscape) after inlining

Note: **C2 JIT** by default does not consider escaping arrays if their size (i.e. the number of elements) is greater than 64

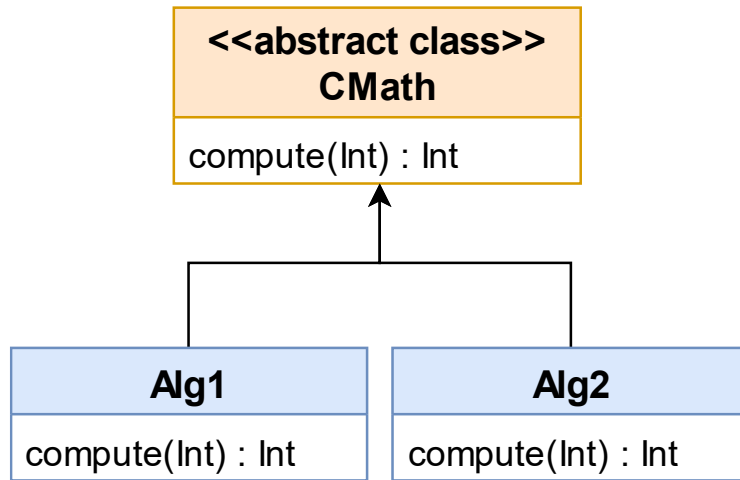
-XX:EliminateAllocationArraySizeLimit=<arraySize>

Virtual Calls

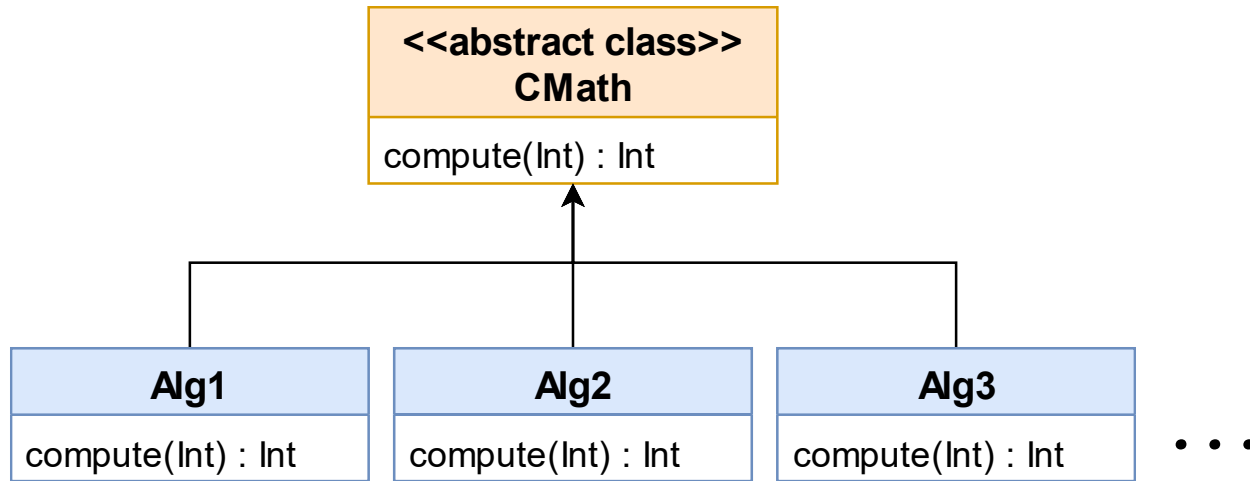
03



Monomorphic call site – optimistically points to **only one concrete method** that has ever been used at that particular call site.



Bimorphic call site – optimistically points to only two concrete methods which can be invoked at a particular call site.



Megamorphic call site – optimistically points to three or possibly more methods which can be invoked at a particular call site.

```
// CMath = {Alg1, Alg2, Alg3, Alg4, Alg5, Alg6}
@CompilerControl(CompilerControl.Mode.DONT_INLINE)
public int execute(CMath cmath, int i) {
    return cmath.compute(i); // param * constant
}
```



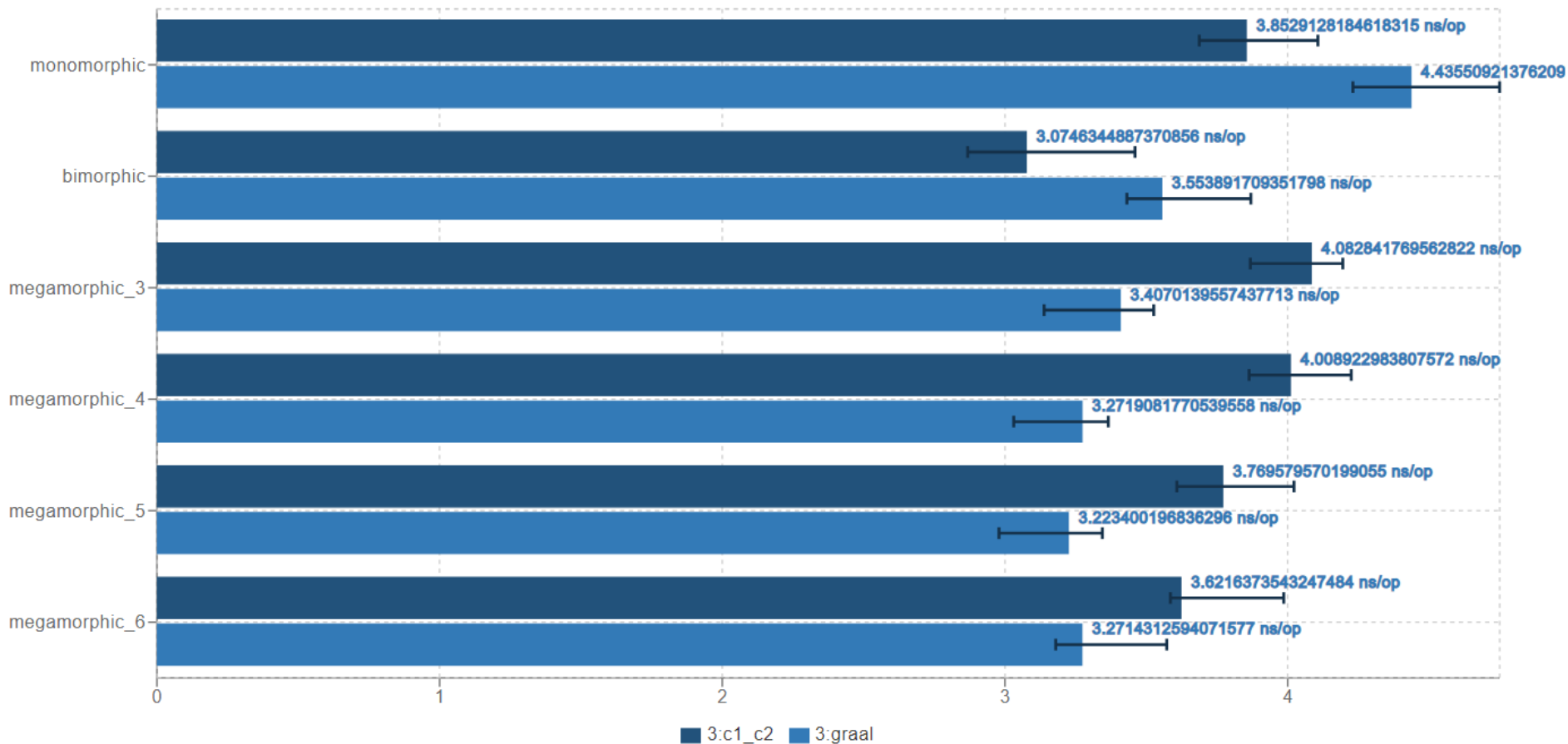
megamorphic call site

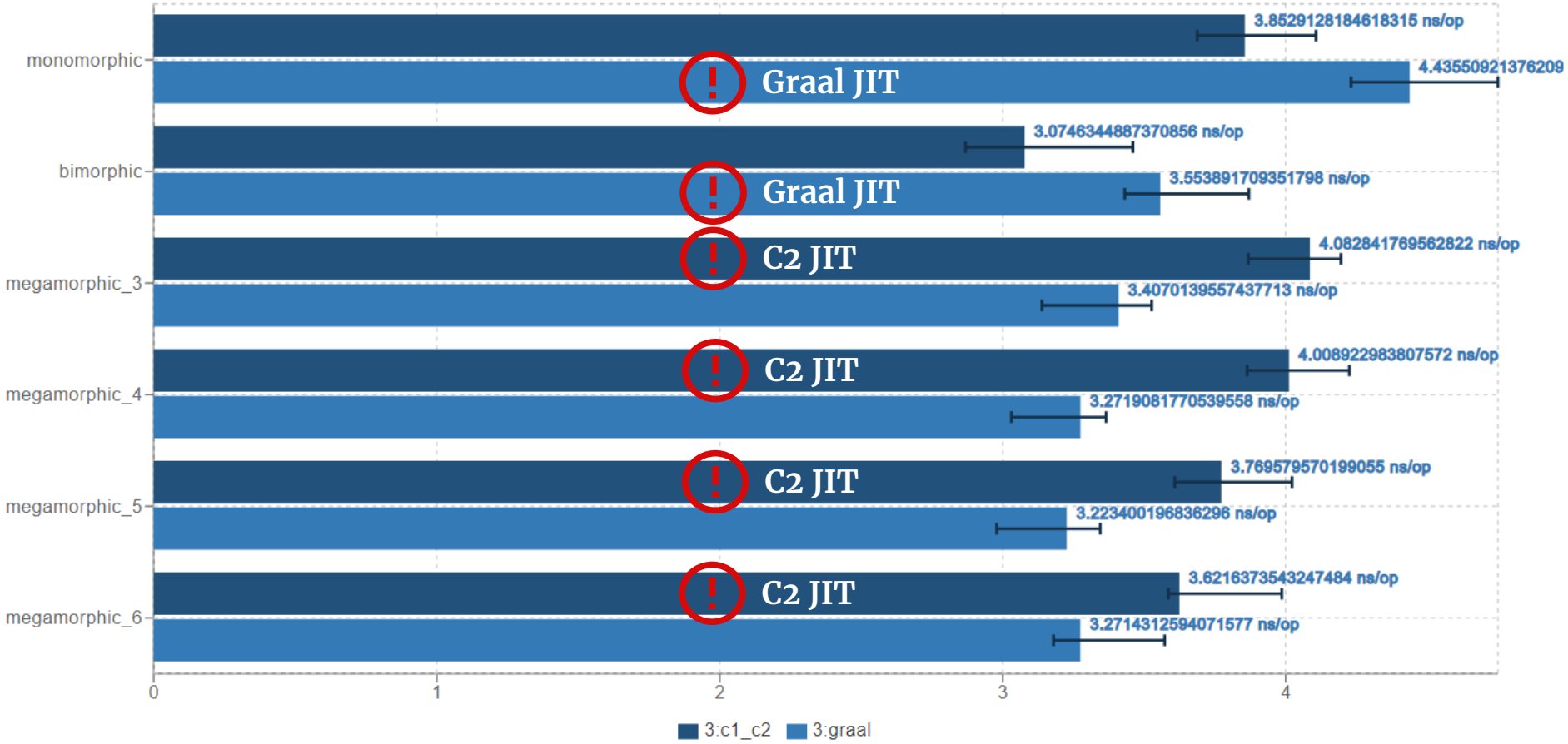

```
@Benchmark
@OperationsPerInvocation(1)
public int monomorphic() {
    return execute(alg1, param)    // param * 17
}
```

```
@Benchmark
@OperationsPerInvocation(2)
public int bimorphic() {
    return execute(alg1, param) + // param * 17
           execute(alg2, param)   // param * 19
}
```

```
@Benchmark
@OperationsPerInvocation(3)
public int megamorphic_3() {
    return execute(alg1, param) + // param * 17
           execute(alg2, param) + // param * 19
           execute(alg3, param)   // param * 23
}

/*
    Similar test cases for:
    megamorphic_4() {...}
    megamorphic_5() {...}
    megamorphic_6() {...}
*/
```





`bimorphic()`
<<under the hood>>

C2 JIT, level 4, execute() ; generated assembly - bimorphic case

```
mov     r10d,DWORD PTR [rsi+0x8]
                                   ; implicit exception

cmp     r10d,0x23757f              ; {metadata(&apos;Alg1&apos;)}
je      L0

cmp     r10d,0x2375c2              ; {metadata(&apos;Alg2&apos;)}
jne     L1
imul    eax,edx,0x13               ; - Alg2::compute

jmp     0x00007f48986678ce         ; return

L0: mov  eax,edx                   ; - Alg1::compute
shl     eax,0x4
add     eax,edx

L1: call 0x00007f4890ba1300        ;*invokevirtual compute
                                   ;{runtime_call UncommonTrapBlob}
```

C2 JIT, level 4, execute() ; generated assembly - bimorphic case

```
mov     r10d,DWORD PTR [rsi+0x8]           ; implicit exception

; {metadata(&apos;Alg1&apos;)}
cmp     r10d,0x23757f
je      L0

; {metadata(&apos;Alg2&apos;)}
cmp     r10d,0x2375c2
jne     L1
imul    eax,edx,0x13                       ; - Alg2::compute

jmp     0x00007f48986678ce                 ; return

; - Alg1::compute
L0: mov  eax,edx
      shl eax,0x4
      add eax,edx

; *invokevirtual compute
; {runtime_call UncommonTrapBlob}
L1: call 0x00007f4890ba1300
```

The diagram illustrates the control flow of the assembly code. A dashed blue line labeled 'Alg1' on the left side starts at the 'je L0' instruction, goes left, then down, and then right to point at the 'L0:' label of the first code block. Another dashed blue line starts at the 'jne L1' instruction, goes left, then down, and then right to point at the 'L1:' label of the second code block.

C2 JIT, level 4, execute() ; generated assembly - bimorphic case

```
mov     r10d,DWORD PTR [rsi+0x8]
```

```
; implicit exception
```

```
cmp     r10d,0x23757f
je      L0
```

```
; {metadata(&apos;Alg1&apos;;)}
```

```
cmp     r10d,0x2375c2
jne     L1
```

```
; {metadata(&apos;Alg2&apos;;)}
```

```
imul    eax,edx,0x13
```

```
; - Alg2::compute
```

```
jmp     0x00007f48986678ce
```

```
; return
```

```
L0: mov     eax,edx
shl     eax,0x4
add     eax,edx
```

```
; - Alg1::compute
```

```
L1: call    0x00007f4890ba1300
```

```
;*invokevirtual compute
```

```
;{runtime_call UncommonTrapBlob}
```


C2 JIT, level 4, execute() ; generated assembly - bimorphic case

```
mov     r10d,DWORD PTR [rsi+0x8]
```

```
; implicit exception
```

```
cmp     r10d,0x23757f
je      L0
```

```
; {metadata(&apos;Alg1&apos;)} }
```

```
cmp     r10d,0x2375c2
jne     L1
imul    eax,edx,0x13
```

```
; {metadata(&apos;Alg2&apos;)} }
```

```
; - Alg2::compute
```

```
jmp     0x00007f48986678ce
```

```
; return
```

```
L0: mov     eax,edx
shl     eax,0x4
add     eax,edx
```

```
; - Alg1::compute
```

```
L1: call    0x00007f4890ba1300
```

```
;*invokevirtual compute
```

```
;{runtime_call UncommonTrapBlob}
```

Graal JIT, execute() ; generated assembly - bimorphic case

```
cmp    rax,QWORD PTR [rip+0xfffffffffffffb8] # 0x00007f4609ed6dc0
je     L0
```

```
cmp    rax,QWORD PTR [rip+0xfffffffffffffb3] # 0x00007f4609ed6dc8
je     L1
```

```
jmp    L2
```

```
L0: imul  eax,edx,0x13          ; - Alg2::compute
ret
```

```
L1: mov   eax,edx
shl     eax,0x4
add     eax,edx              ; - Alg1::compute
ret
```

```
L2: call  0x00007f45ffba10a4    ;*invokevirtual compute
                                   ;{runtime_call DeoptimizationBlob}
```

Graal JIT, execute() ; generated assembly - bimorphic case

```
cmp    rax,QWORD PTR [rip+0xffffffffffffffb8] # 0x00007f4609ed6dc0
je     L0
```

```
cmp    rax,QWORD PTR [rip+0xffffffffffffffb3] # 0x00007f4609ed6dc8
je     L1
```

```
jmp    L2
```

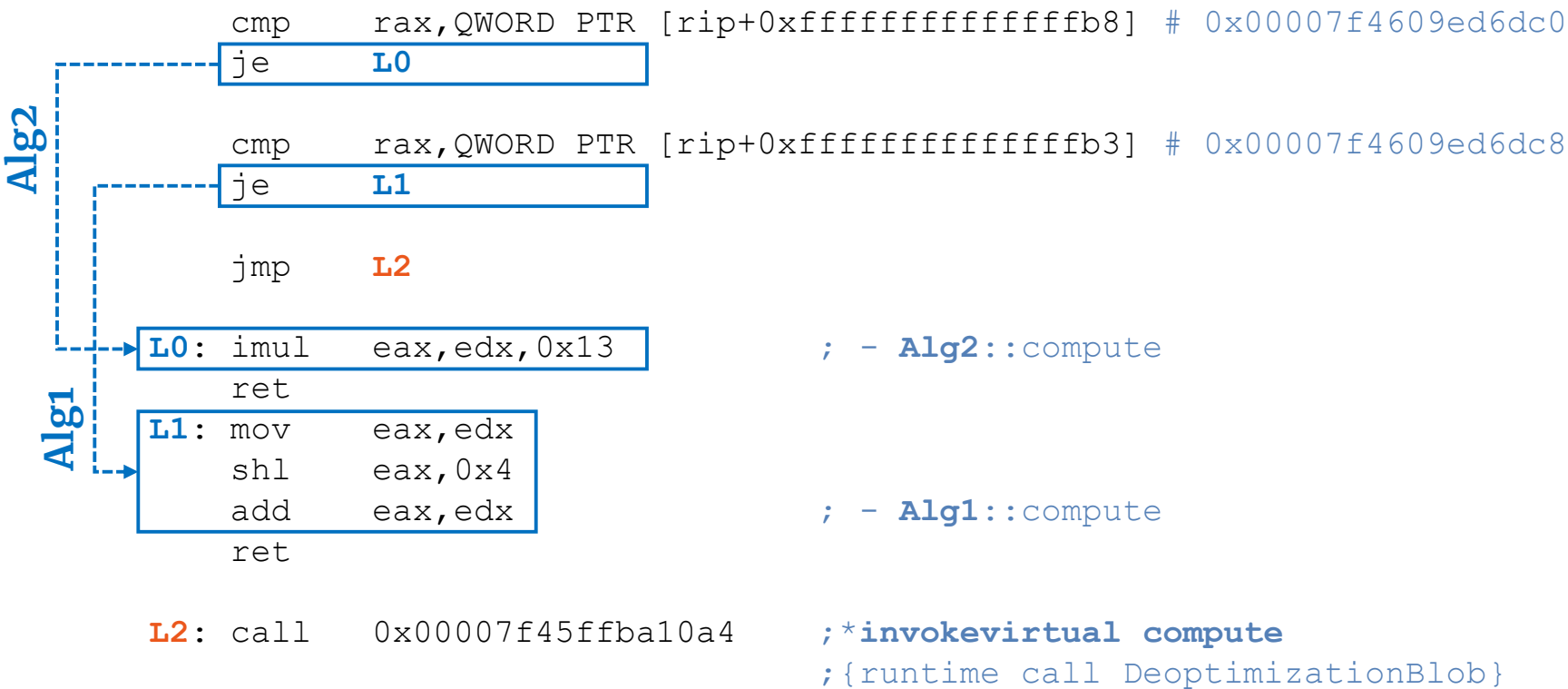
```
L0: imul    eax,edx,0x13                ; - Alg2::compute
ret
```

```
L1: mov     eax,edx
shl     eax,0x4
add     eax,edx                ; - Alg1::compute
ret
```

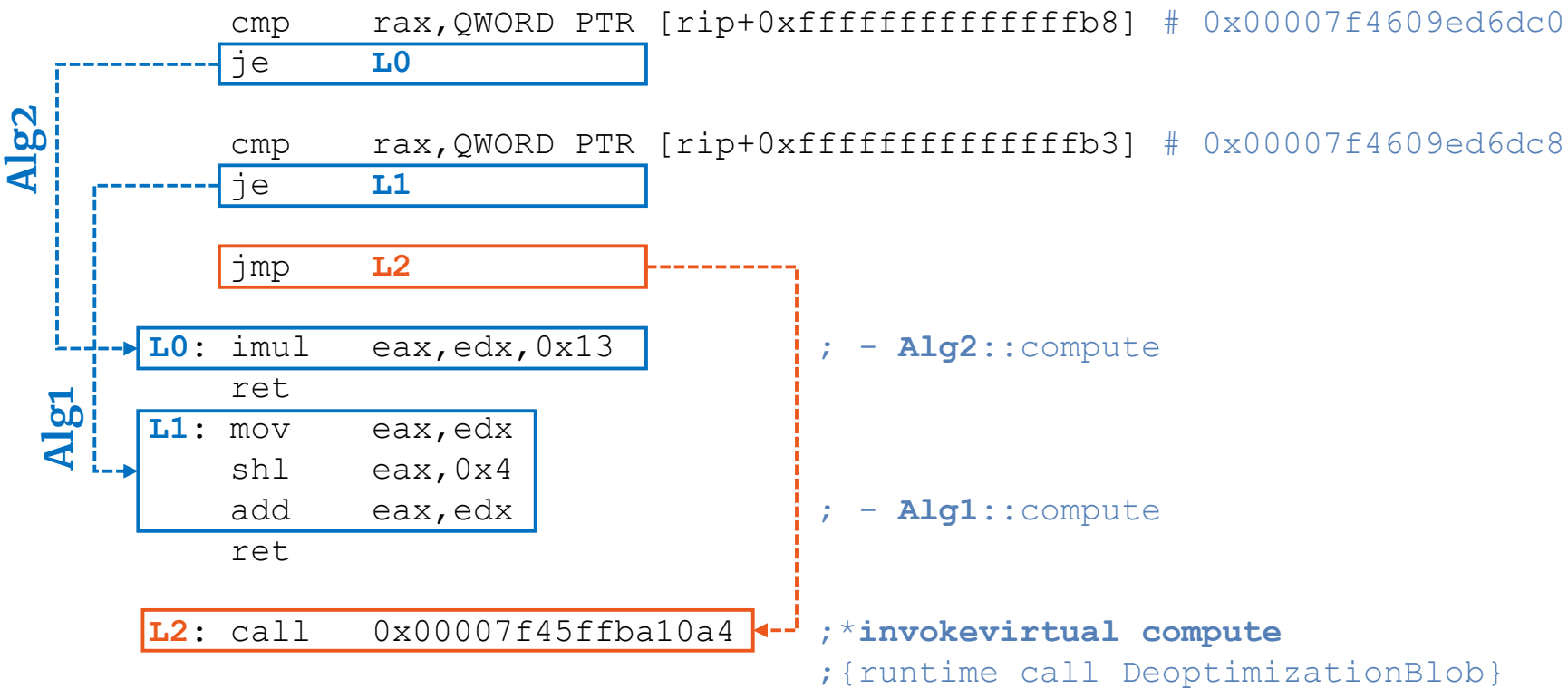
```
L2: call    0x00007f45ffba10a4        ;*invokevirtual compute
                                         ;{runtime_call DeoptimizationBlob}
```

Alg1

Graal JIT, execute() ; generated assembly - bimorphic case



Graal JIT, execute() ; generated assembly - bimorphic case



Conclusions - bimorphic call site

Both compilers (**Graal JIT** and **C2 JIT**) optimize bimorphic call sites in a similar approach:

- inline caching plus several comparison/jump checks
- adds a deoptimization routine (Graal JIT) or an uncommon trap (C2 JIT) for another, unknown, possible target invocation

megamorphic_3()
<<under the hood>>

C2 JIT, level 4, execute() ; generated assembly - megamorphic_3 case

```
movabs rax,0xfffffffffffffffff
call    0x00007fd65cb9fb80      ;*invokevirtual compute
                                   ;{virtual_call}
```

 Not any further optimization.

Graal JIT, execute() ; generated assembly - megamorphic_3 case

```
cmp    rax,QWORD PTR [rip+0xfffffffffffffb8] # 0x00007fb849ee0bc0
je     L0

cmp    rax,QWORD PTR [rip+0xfffffffffffffb3] # 0x00007fb849ee0bc8
je     L1

cmp    rax,QWORD PTR [rip+0xfffffffffffffae] # 0x00007fb849ee0bd0
je     L2

jmp    L3

L0: ...                               ; - Alg1::compute

L1: ...                               ; - Alg3::compute

L2: ...                               ; - Alg2::compute

L3: call    0x00007fb83fba10a4         ; *invokevirtual compute
                                         ; {runtime_call DeoptimizationBlob}
```

Graal JIT, execute() ; generated assembly - megamorphic_3 case

Alg1

```
cmp    rax,QWORD PTR [rip+0xfffffffffffffb8] # 0x00007fb849ee0bc0
```

```
je     L0
```

```
cmp    rax,QWORD PTR [rip+0xfffffffffffffb3] # 0x00007fb849ee0bc8
```

```
je     L1
```

```
cmp    rax,QWORD PTR [rip+0xfffffffffffffae] # 0x00007fb849ee0bd0
```

```
je     L2
```

```
jmp    L3
```

```
L0: ...
```

```
;- Alg1::compute
```

```
L1: ...
```

```
;- Alg3::compute
```

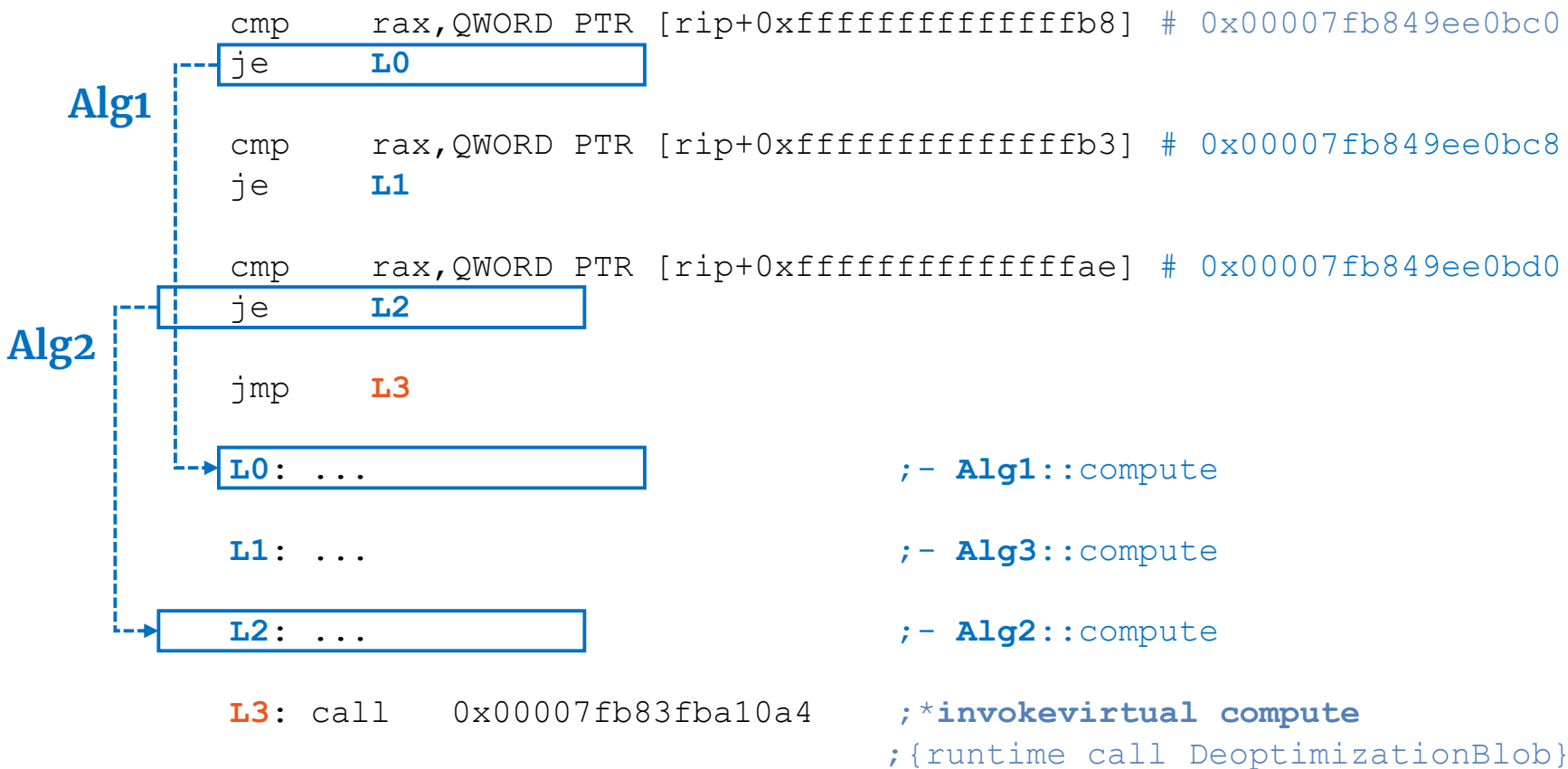
```
L2: ...
```

```
;- Alg2::compute
```

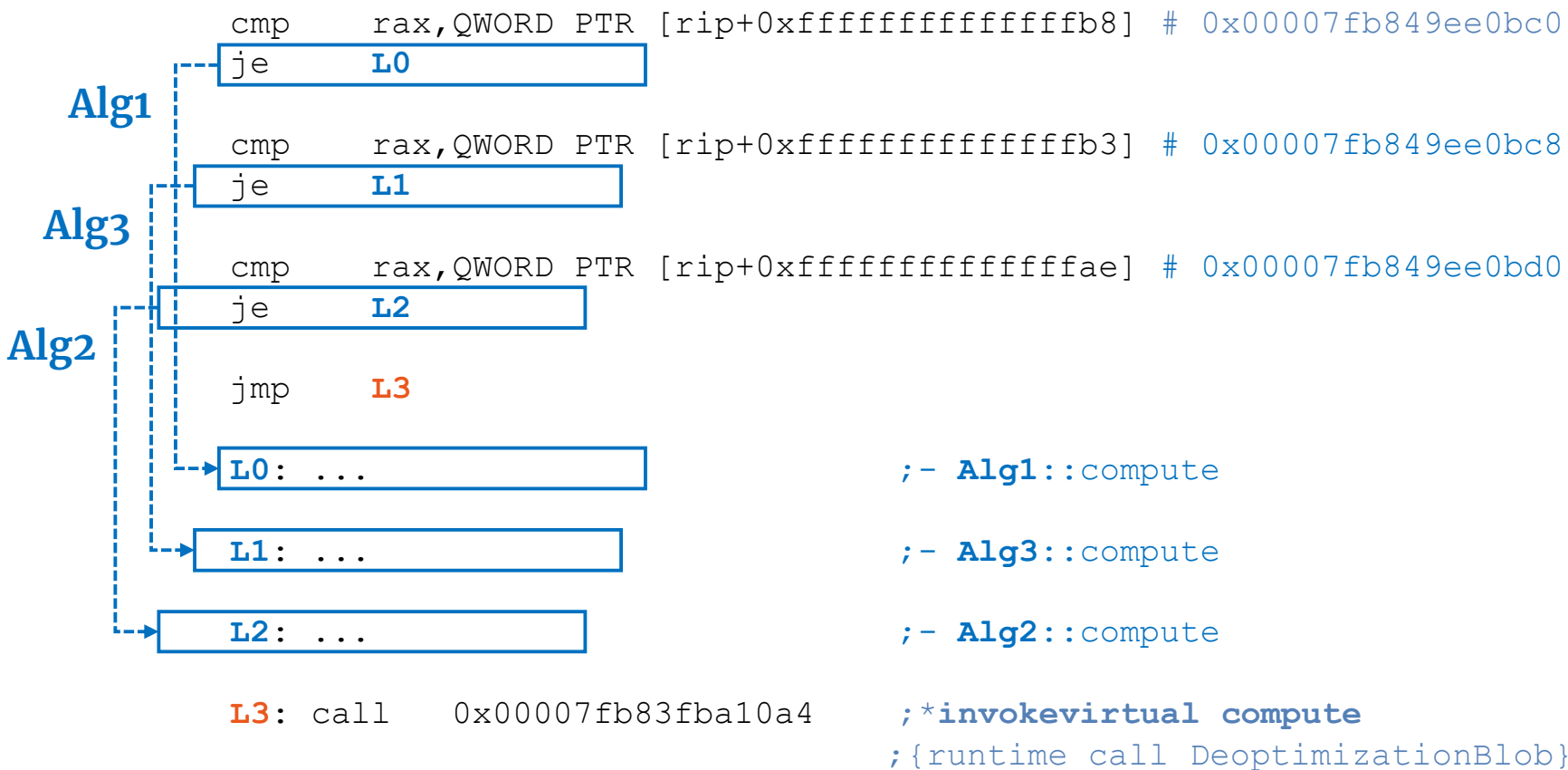
```
L3: call 0x00007fb83fba10a4
```

```
;*invokevirtual compute  
;{runtime_call DeoptimizationBlob}
```

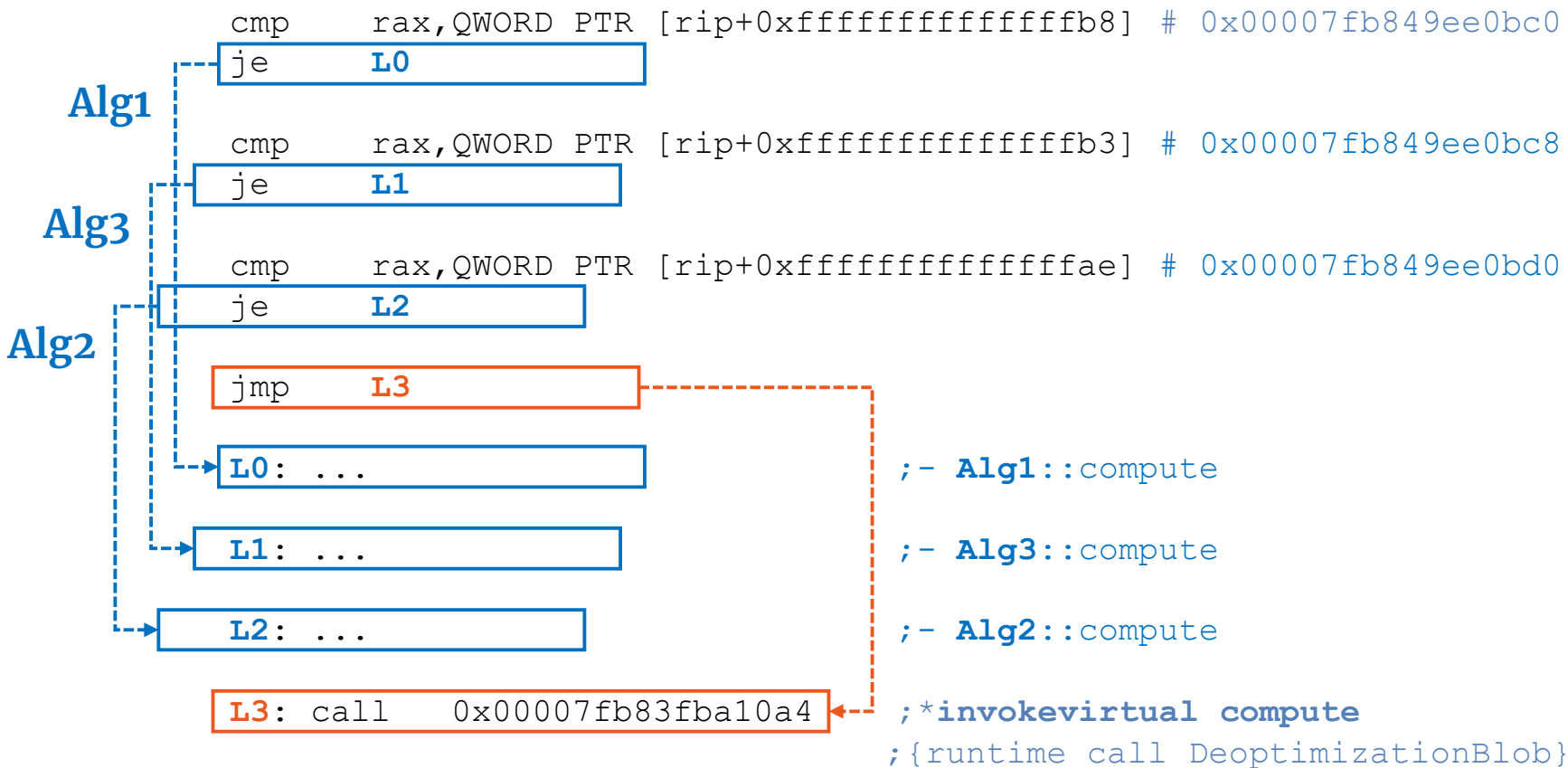
Graal JIT, execute() ; generated assembly - megamorphic_3 case



Graal JIT, execute() ; generated assembly - megamorphic_3 case



Graal JIT, execute() ; generated assembly - megamorphic_3 case



Conclusions - megamorphic_3 call site

Graal JIT optimizes three target method calls as follows:

- inline caching plus several comparison/jump checks
- adds a deoptimization routine for another (unknown) possible target invocation.

Conclusions - megamorphic_3 call site

Graal JIT optimizes three target method calls as follows:

- inline caching plus several comparison/jump checks
- adds a deoptimization routine for another (unknown) possible target invocation.

C2 JIT does not perform any further optimization, there is a virtual call (e.g. vtable lookup) → due to “**polluted profile**” context (i.e. method is used in many different contexts with independent operand types).



JDK / JDK-8015416

tier one should collect context-dependent split profiles

Details

Type:	Enhancement	Status:	OPEN
Priority:	P3	Resolution:	Unresolved
Affects Version/s:	9, 10	Fix Version/s:	tbd
Component/s:	hotspot		
Labels:			
Subcomponent:	compiler		

Description

To avoid polluted profiles, tier one should create disjoint "split" profiles for methods that it inlines. If a method is inlined three times, it should get three fresh copies of its global profile, initialized to no types or counts. When a later tier re-optimizes the code and inlines the same method, the compiler should use the context-dependent profile in preference to the global profile, if one is available.

Background:

The interpreter collects type profiles for invoke receivers and type checks. The profiles are crucial to later optimizing compilation. Tier one currently emulates the interpreter with respect to profiling.

People

Assignee:	Unassigned
Reporter:	John Rose
Votes:	0 Vote for this issue
Watchers:	4 Start watching this issue

Dates

Created:	2013-05-24 19:55
Updated:	2018-10-04 23:10

Conclusions - megamorphic_3 call site

Graal JIT optimizes three target method calls as follows:

- inline caching plus several comparison/jump checks
- adds a deoptimization routine for another (unknown) possible target invocation.

C2 JIT does not perform any further optimization, there is a virtual call (e.g. vtable lookup) → due to “**polluted profile**” context (i.e. method is used in many different contexts with independent operand types).

Note: **megamorphic_4**, **megamorphic_5**, and **megamorphic_6** are optimized in similar ways by **Graal JIT** and **C2 JIT**

Vectorization & Loop Optimizations

04

Scenario: $C[i] = A[i] \times B[i]$

Scalar version processes
a single pair of operands
at a time.

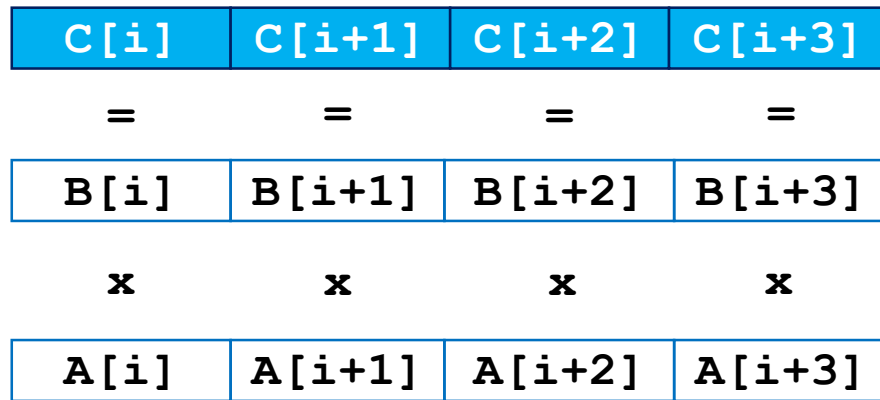
$$\begin{array}{c} \boxed{C[i]} \\ = \\ \boxed{B[i]} \\ \times \\ \boxed{A[i]} \end{array}$$

Scenario: $C[i] = A[i] \times B[i]$

Scalar version processes a single pair of operands at a time.



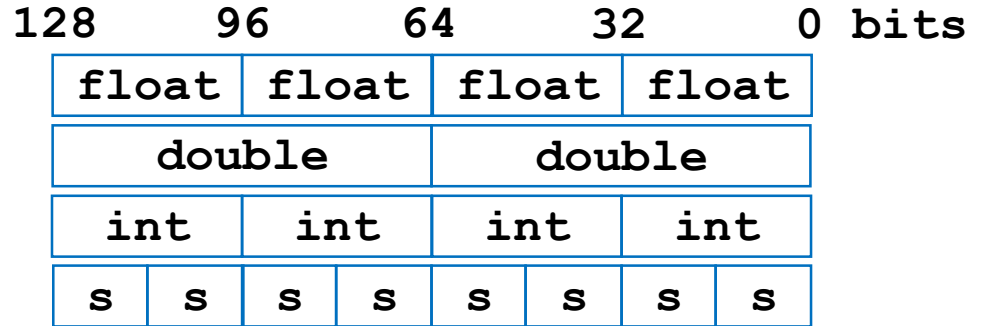
Vectorized version carries out the same instructions on multiple pairs of operands at once.



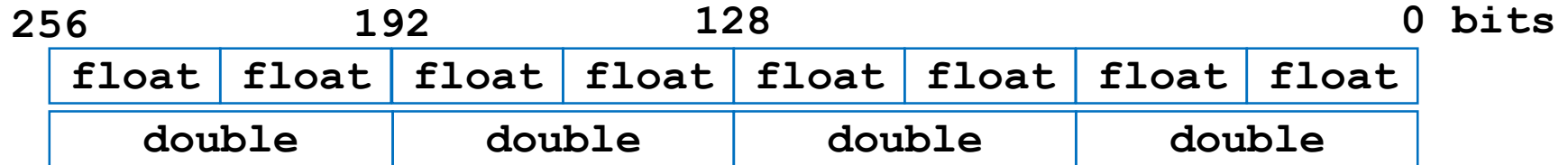
Vector operations implemented via SIMD (Single Instruction Multiple Data) ops.

x86 SIMD extensions

XMM registers



YMM registers



Note: few registers mentioned in current presentation. Check Intel manuals for a complete list.

Modern compilers analyze loops in serial code to identify if they could be vectorized (i.e. perform loop transformations) - **auto vectorization**

Loops requirements for auto vectorization:

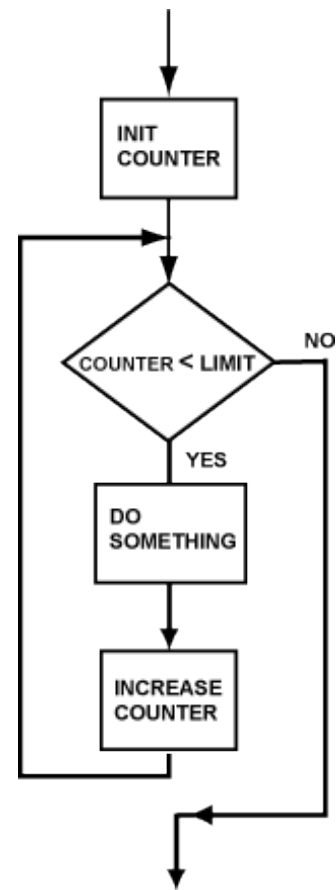
- countable loops → only unrolled loops
- single entry and exit
- straight-line code (e.g. no switch, if with masking is allowed)
- only for most inner loop (caution in case of loop interchange or loop collapsing)
- no functions calls, only intrinsics are allowed

Countable Loops

```
1) for (int i = start; i < limit; i += stride) {  
    // loop body  
}
```

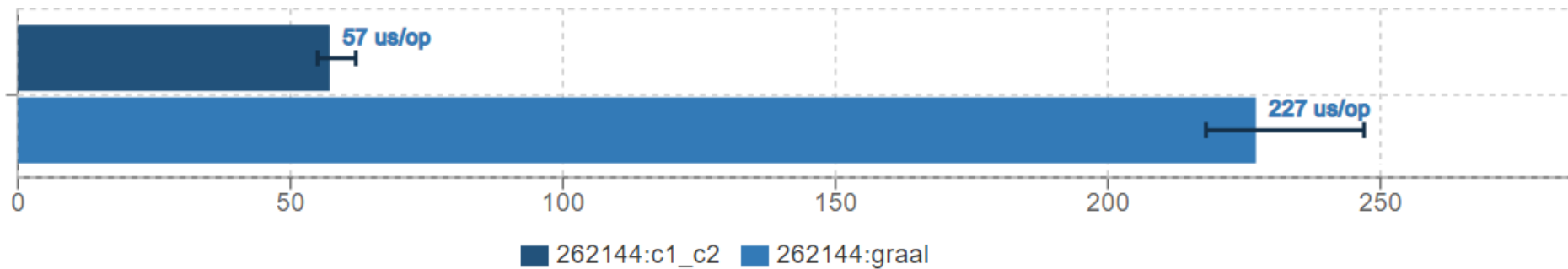
```
2) int i = start;  
   while (i < limit) {  
       // loop body  
       i += stride;  
   }
```

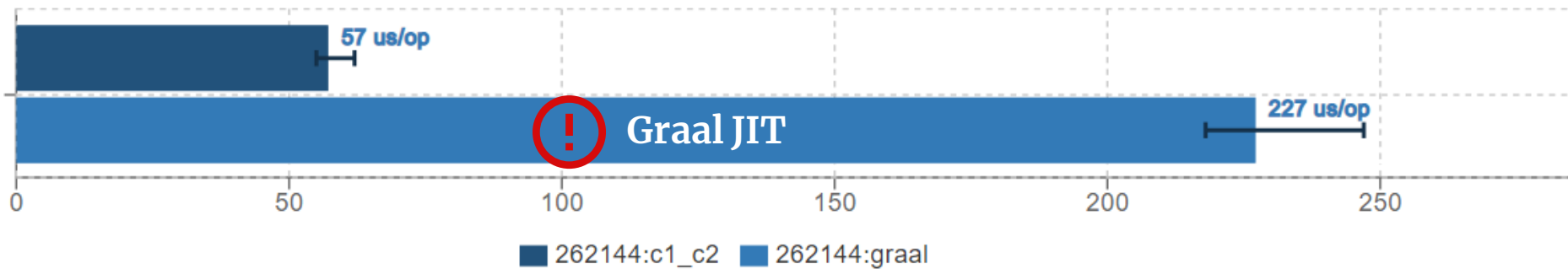
-
- **limit** is loop invariant
 - **stride** is constant (compile-time known)



$$C[i] = A[i] \times B[i]$$


```
@Param({ "262144" })  
private int size;  
private float[] A, B, C;  
  
@Benchmark  
public float[] multiply_2_arrays_elements() {  
    for (int i = 0; i < size; i++) {  
        C[i] = A[i] * B[i];  
    }  
    return C;  
}
```





C2 JIT loop vectorization pattern

Scalar pre-loop

alignment (CPU caches)

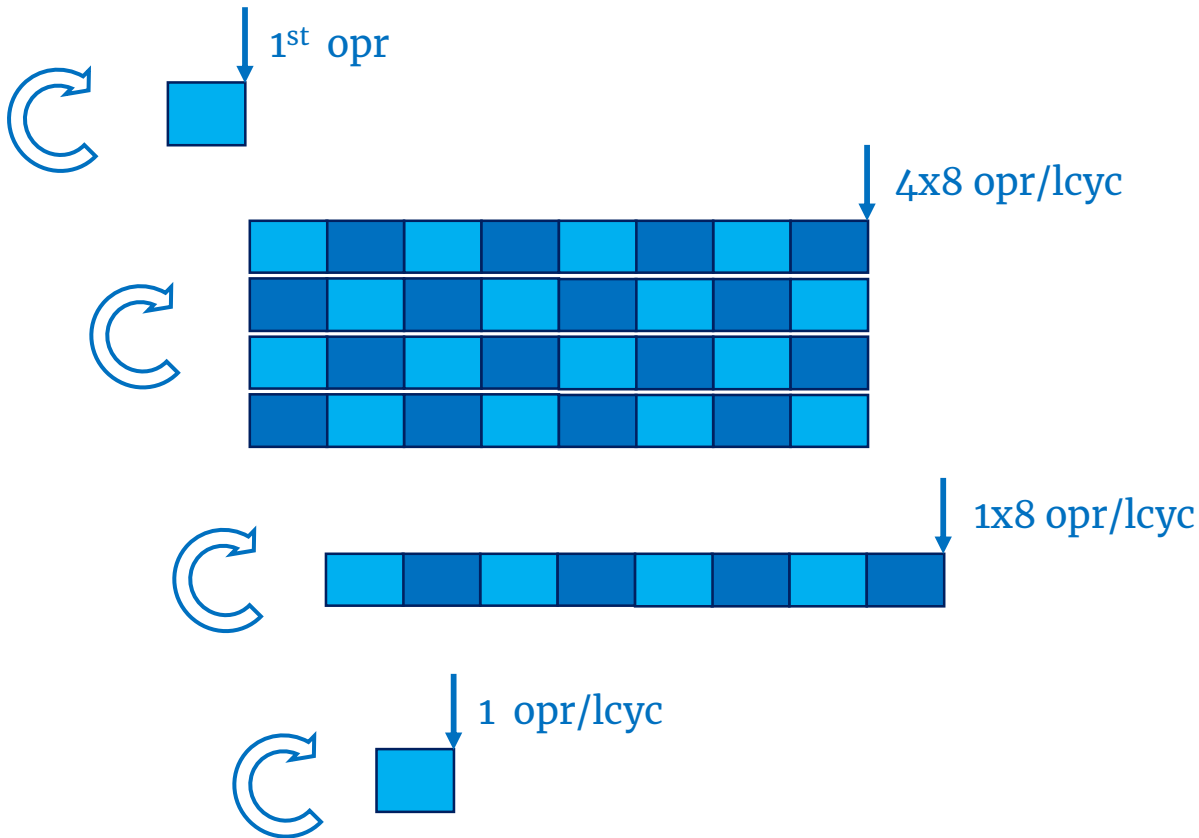
Vectorized main-loop

e.g. YMM registers

Vectorized post-loop

e.g. YMM registers

Scalar post-loop



Legend: opr → loop body (i.e. operand(s) / array element(s)); lcyc → loop cycle

C2 JIT, **Scalar pre-loop** (1 float operation/loop cycle)

L0000:

```
    vmovss    xmm2,DWORD PTR [rdi+r13*4+0x10]           ;*faload
    vmulss    xmm2,xmm2,DWORD PTR [rsi+r13*4+0x10]      ;*fmul
    vmovss    DWORD PTR [rax+r13*4+0x10],xmm2          ;*fastore
    inc       r13d                                       ;*iinc
    cmp       r13d,ebx
    jl  L0000                                           ;*if_icmpge
```

```
/* Pattern: C[i] = A[i] x B[i]; i+=1 */
```

C2 JIT, **Vectorized main-loop** (4x8 float operations/loop cycle)

L0001:

```
vmovdqu ymm2, YMMWORD PTR [rdi+r13*4+0x10]
vmulps ymm2, ymm2, YMMWORD PTR [rsi+r13*4+0x10]
vmovdqu YMMWORD PTR [rax+r13*4+0x10], ymm2
movsxd rcx, r13d
...
vmovdqu ymm2, YMMWORD PTR [rdi+rcx*4+0x70]
vmulps ymm2, ymm2, YMMWORD PTR [rsi+rcx*4+0x70]
vmovdqu YMMWORD PTR [rax+rcx*4+0x70], ymm2           ;*fastore
add r13d, 0x20                                     ;*iinc
cmp r13d, r9d
j1 L0001                                           ;*if_icmpge

/* Pattern: C[i:i+7] = A[i:i+7] x B[i:i+7]; i+=32 */
```

C2 JIT, Vectorized post-loop (8 float operations/loop cycle)

L0002:

```
vmovdqu ymm2, YMMWORD PTR [rdi+r13*4+0x10]
vmulps   ymm2, ymm2, YMMWORD PTR [rsi+r13*4+0x10]
vmovdqu YMMWORD PTR [rax+r13*4+0x10], ymm2      ;*fastore
add      r13d, 0x8                               ;*iinc
cmp      r13d, r9d
jl L0002                                         ;*if_icmpge
```

```
/* Pattern: C[i:i+7] = A[i:i+7] x B[i:i+7]; i+=8 */
```

C2 JIT, Scalar post-loop (1 float operation/loop cycle)

L0004:

```
    vmovss    xmm4,DWORD PTR [rdi+r13*4+0x10]           ;*faload
    vmulss    xmm4,xmm4,DWORD PTR [rsi+r13*4+0x10]      ;*fmul
    vmovss    DWORD PTR [rax+r13*4+0x10],xmm4          ;*fastore
    inc       r13d                                       ;*iinc
    cmp       r13d,r11d
    jl  L0004                                           ;*iload_1
```

```
/* Pattern: C[i] = A[i] x B[i]; i+=1 */
```

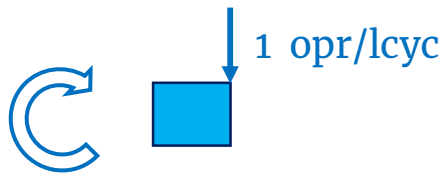

Graal JIT loop optimization pattern

Loop peeling

alignment (CPU caches)



Scalar loop



Lack of **loop vectorization** support.
No **loop unrolling**.

Legend: opr → loop body (i.e. operand(s) / array element(s)); lcyc → loop cycle

Graal JIT, **Loop peeling** (2 float operations)

```
shl      rcx,0x3                                ;*getfield B
vmovss   xmm0,DWORD PTR [rcx+r13*4+0x10]        ;*faload
shl      r8,0x3                                ;*getfield A
vmulss   xmm0,xmm0,DWORD PTR [r8+r13*4+0x10]    ;*fmul
vmovss   DWORD PTR [r11+r13*4+0x10],xmm0        ;*fastore
mov      edx,r13d
inc      edx                                    ;*iinc

vmovss   xmm0,DWORD PTR [rcx+rdx*4+0x10]        ;*faload
vmulss   xmm0,xmm0,DWORD PTR [r8+rdx*4+0x10]    ;*fmul
vmovss   DWORD PTR [r11+rdx*4+0x10],xmm0        ;*fastore

lea      edx,[r13+0x2]                          ;*iinc
jmp      L0001                                ;main_loop
```

Graal JIT, **Scalar loop** (1 float operation/loop cycle)

L0000:

```
vmovss xmm0,DWORD PTR [rcx+rdx*4+0x10]      ;*faload
vmulss xmm0,xmm0,DWORD PTR [r8+rdx*4+0x10]  ;*fmul
vmovss DWORD PTR [r11+rdx*4+0x10],xmm0      ;*fastore
inc     edx                                  ;*iinc
```

L0001:

```
cmp     r10d,edx
jg L0000                                     ;*if_icmpge
```

```
/* Pattern: C[i] = A[i] x B[i]; i+=1 */
```

Conclusions – multiply_2_arrays_elements

C2 JIT uses vectorization for:

- **main loop** - YMM AVX2 registers → 4x8 float operations/loop cycle
- **post loop** - YMM AVX2 registers → 8 float operations/loop cycle

C2 JIT handles the **remaining post loop** without unrolling and without vectorization, just one by one until reaches the end

Conclusions – multiply_2_arrays_elements

C2 JIT uses vectorization for:

- **main loop** - YMM AVX2 registers → 4x8 float operations/loop cycle
- **post loop** - YMM AVX2 registers → 8 float operations/loop cycle

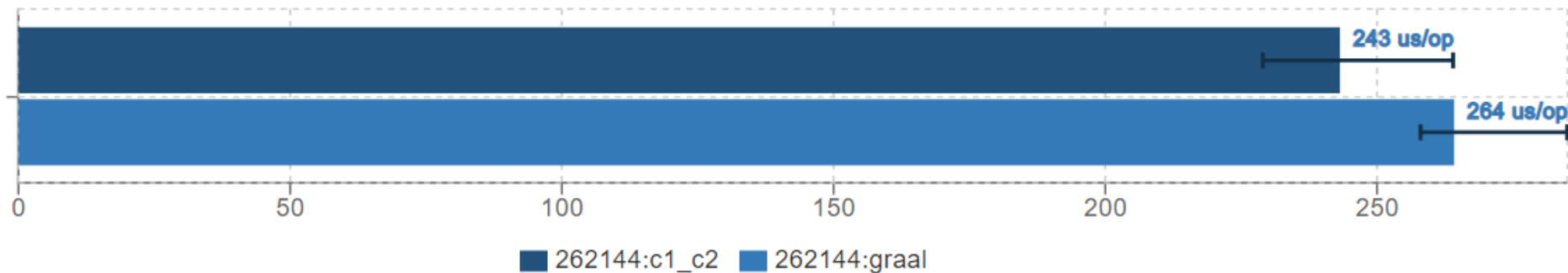
C2 JIT handles the **remaining post loop** without unrolling and without vectorization, just one by one until reaches the end

Graal JIT has no vectorization support → core feature missing in OpenJDK!

Graal JIT does not trigger loop unrolling (in this particular case)

```
C[1] = A[1] x B[1]  
    <<stride:long>>
```

```
@Param({ "262144" })  
private int size;  
private float[] A, B, C;  
  
@Benchmark  
public float[] multiply_2_arrays_elements_long_stride() {  
    for (long l = 0; l < size; l++) {  
        C[(int) l] = A[(int) l] * B[(int) l];  
    }  
    return C;  
}
```

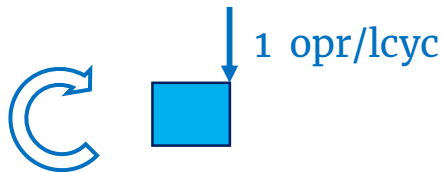


Note: the average timings for previous case (i.e. **int stride**) were:

- ♦ **C2 JIT** - 57 us/op (i.e. ~4.2x faster)
- ♦ **Graal JIT** - 227 us/op (i.e. ~1.2x faster)

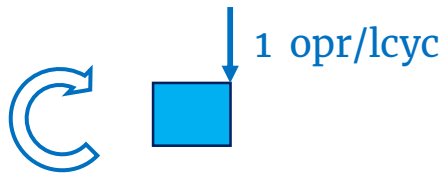
C2 JIT loop optimization pattern

Scalar loop



Graal JIT loop optimization pattern

Scalar loop



Lack of **loop vectorization** support.
No **loop unrolling**.

C2 JIT, Scalar loop (1 float operation/loop cycle)

L0000:

```
vmovss DWORD PTR [rsi+rcx*4+0x10],xmm1           ;*goto
mov     rcx,QWORD PTR [r15+0x108]
add     r13,0x1                                   ;*ladd
test    DWORD PTR [rcx],eax                       ;* SAFEPPOINT POLL * 
cmp     r13,rdx
jge     L0002                                     ;*ifge
mov     ecx,r13d                                  ;*l2i
vmovss  xmm1,DWORD PTR [rax+rcx*4+0x10]           ;*faload
vmulss  xmm1,xmm1,DWORD PTR [r14+rcx*4+0x10]       ;*fmul
cmp     ecx,r9d
jb  L0000

/* Pattern: C[i] = A[i] x B[i]; i+=1 */
```

Graal JIT, **Scalar loop** (1 float operation/loop cycle)

L0000:

```
mov     esi,ecx                                ;*l2i
vmovss  xmm0,DWORD PTR [rdi+rsi*4+0x10]        ;*faload
vmulss  xmm0,xmm0,DWORD PTR [r8+rsi*4+0x10]    ;*fmul
vmovss  DWORD PTR [r11+rsi*4+0x10],xmm0        ;*fastore

mov     rsi,QWORD PTR [r15+0x108]              ;*lload_1
test    DWORD PTR [rsi],eax                   ;* SAFEPPOINT POLL * 
inc     rcx                                   ;*ladd
cmp     r10,rcx

jg L0000                                       ;*ifge

/* Pattern: C[i] = A[i] x B[i]; i+=1 */
```

Conclusions – multiply_2_arrays_elements_long_stride

Both compilers (**Graal JIT** and **C2 JIT**) optimizes the float arrays elements multiplication with long stride in a similar fashion:

- one main scalar loop
- no loop unrolling
- no vectorization
- a safepoint poll is added within the loop

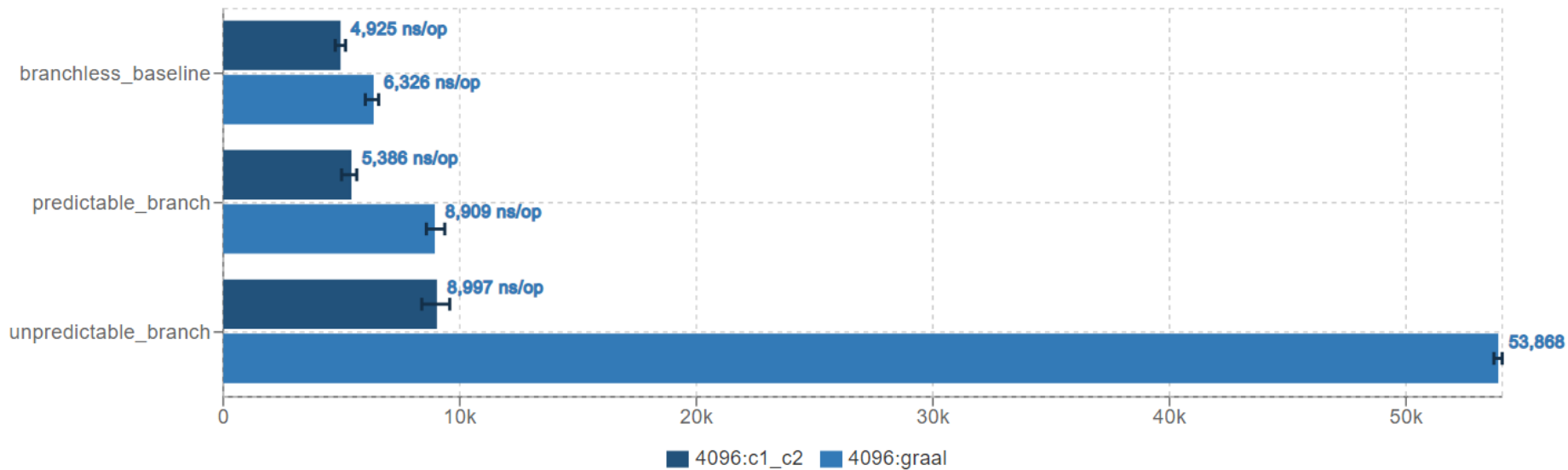
`unpredictable_branch()`

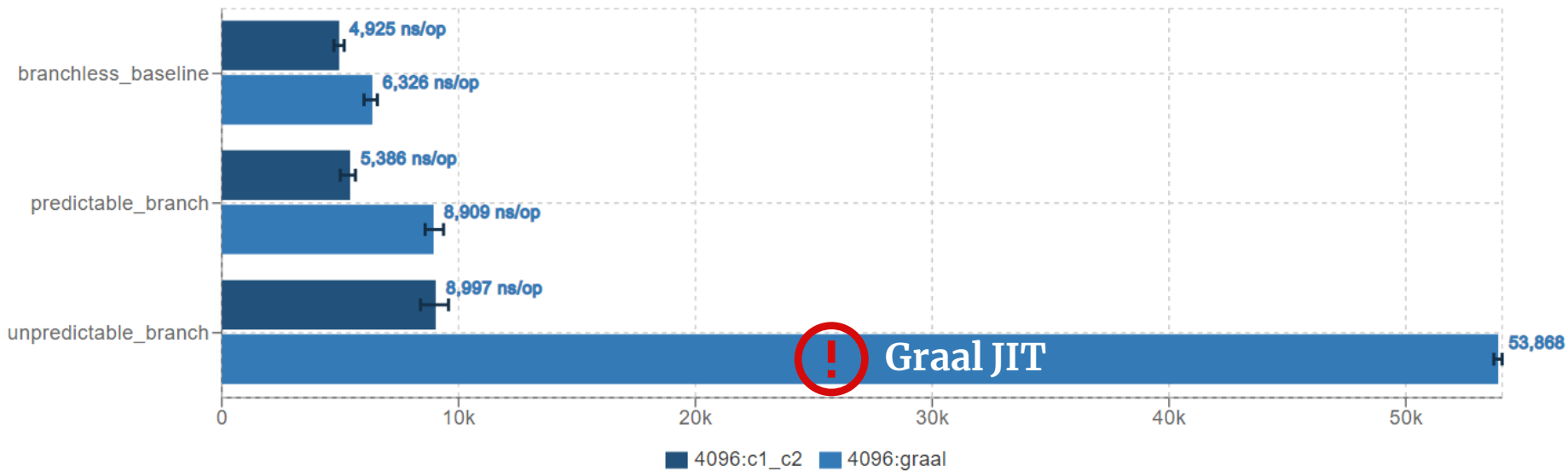
```
@Param({"4096"}) private int thresholdLimit;
private final int SIZE = 16_384;
private int[] array = new int[SIZE];

// Initialize array elements with values between [0, thresholdLimit)
// e.g. array[i] = random.nextInt(thresholdLimit); // i = [0, SIZE)

@Benchmark
public int unpredictable_branch() {
    int sum = 0;

    for (final int value : array) {
        if (value <= (thresholdLimit / 2)) {
            sum += value;
        }
    }
    return sum;
}
```

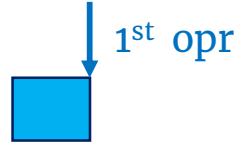




C2 JIT loop optimization pattern

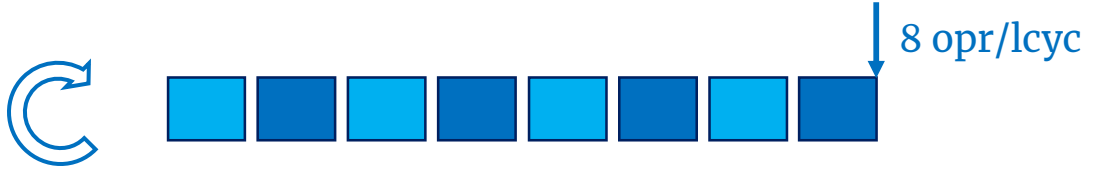
Loop peeling

alignment (CPU caches)

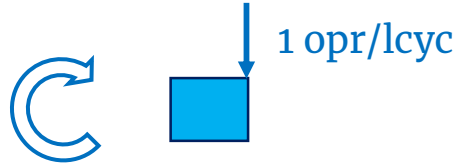


Scalar main-loop

e.g. unrolling factor = 8



Scalar post-loop



Legend: opr → loop body (i.e. operand(s) / array element(s)); lcyc → loop cycle

C2 JIT, Loop peeling (1st array element)

```
mov     ebp,DWORD PTR [rsi+0x14]
mov     r10d,DWORD PTR [r12+rbp*8+0xc]
mov     eax,DWORD PTR [r12+rbp*8+0x10]
mov     ecx,DWORD PTR [rsi+0x10]
mov     edi,0x1
```

```
;*getfield array
;*arraylength
;*iaload
;*getfield thresholdLimit
;main loop start index
```

C2 JIT, Scalar main-loop (8 array elements/loop cycle)

L0000:

```
mov    r10d,DWORD PTR [rbp+rdi*4+0x10]    ;*iaload
mov    r9d,DWORD PTR [rbp+rdi*4+0x14]
mov    r8d,DWORD PTR [rbp+rdi*4+0x18]
mov    ebx,DWORD PTR [rbp+rdi*4+0x1c]
mov    ecx,DWORD PTR [rbp+rdi*4+0x20]
mov    edx,DWORD PTR [rbp+rdi*4+0x24]
mov    esi,DWORD PTR [rbp+rdi*4+0x28]
mov    r11d,eax
add    r11d,r10d
cmp    r10d,r13d                        ; r13d = thresholdLimit/2
cmovle eax,r11d                        ;*iinc
mov    r10d,DWORD PTR [rbp+rdi*4+0x2c]    ;*iaload
...
add    edi,0x8                          ;*iinc
cmp    edi,r14d
jl     L0000                            ;*if_icmpge
```

C2 JIT, Scalar post-loop (1 array element/loop cycle)

L0001:

```
    mov     r11d,DWORD PTR [rbp+rdi*4+0x10]           ;*iaload

    mov     r9d,eax
    add     r9d,r11d
    cmp     r11d,r13d                                ; r13d = thresholdLimit/2
    cmovle  eax,r9d

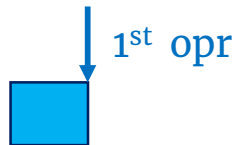
    inc     edi                                       ;*iinc
    cmp     edi,r10d

    jl      L0001                                    ;*if_icmpge
```

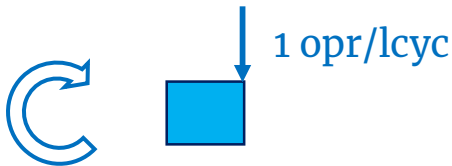
Graal JIT loop optimization pattern

Loop peeling

alignment (CPU caches)



Scalar loop



! No **loop unrolling**.

Legend: opr → loop body (i.e. operand(s) / array element(s)); lcyc → loop cycle

Graal JIT, **Loop peeling** (1st array element)

```
mov    r11d,DWORD PTR [rsi+0x10]
mov    r8d,DWORD PTR [rax*8+0x10]
shl     rax,0x3
mov    r11d,0x1
```

```
;*getfield thresholdLimit
;*iaload
;*getfield array
```

Graal JIT, **Scalar loop** (1 array element/loop cycle)

L0000:

mov	ecx,DWORD PTR [rax+r11*4+0x10]	;*iaload
cmp	ecx,r9d	; r13d = thresholdLimit/2
jg	L0003	;*if_icmpgt
add	r8d,ecx	;*iadd
inc	r11d	;*iinc
L0001:	cmp r10d,r11d	
jg	L0000	;*if_icmpge

Conclusions – unpredictable_branch

C2 JIT optimizes the loop as follows:

- **main loop** - with an unrolling factor of 8
- **post loop** - one by one, until it reaches the end

Conclusions – unpredictable_branch

C2 JIT optimizes the loop as follows:

- **main loop** - with an unrolling factor of 8
- **post loop** - one by one, until it reaches the end

Graal JIT does not unroll the loop (second example without unrolling!)

Summary & Takeaways

05

	Graal JIT	C2 JIT
Scalar Replacement	✓	
Virtual Calls	✓	
Vectorization	!	✓
Loop Unrolling		✓
Intrinsics		✓

Scalar Replacement: nice optimizations by Graal JIT (e.g. Partial Escape Analysis).

Virtual Calls: better optimized by Graal JIT (e.g. megamorphic call sites).

Vectorization: core compiler feature “missing” in Graal JIT (OpenJDK).

Loop unrolling: not found in current Graal JIT test cases. Missing or just limited!?

Intrinsics: not covered here but the support is lower in Graal JIT. See reference.

Application Developer Guidelines

	Graal JIT	C2 JIT
a lot of objects created	✓	
high degree of polymorphic calls	✓	
myriad of tiny nested calls (i.e. functional programming)	✓	
a lot of computational loops / mathematical operations		✓
quicker JIT start-up performance / lower JIT runtime overhead		✓

Note: please take these guidelines with precaution, they are not general valid!

“Measure, do not guess” and apply what fits your application/context.

Open JDK support

Version	Graal JIT	C2 JIT
8	❗ [1]	✅
9	❗ [2]	✅
10	✅	✅
11	✅	✅
12	✅	✅
13	✅	✅

[\[1\]](#) – for Java 8 there are backports available

[\[2\]](#) – for Java 9 Graal JIT needs to be manually added on top of the OpenJDK distribution

Garbage Collectors compatibility in Open JDK

	Graal JIT	C2 JIT
Serial	✓	✓
Parallel/ParallelOld	✓	✓
CMS	!	✓
G1	✓	✓
Z	!	✓
Epsilon	!	✓
Shenandoah ^[1]	!	✓

^[1] – included in RedHat OpenJDK builds

Note: JVMCI Compiler does not support selected GC: {CMS, Z; Shenandoah; Epsilon}



◆ **Thanks**

Resources

Slides

[IonutBalosin.com/talks](https://ionutbalosin.com/talks)

Further Readings

1. <https://ionutbalosin.com/2019/04/jvm-jit-compilers-benchmarks-report-19-04>
2. <https://graalworkshop.github.io/2019/Performance-Characterization-And-Optimizations-In-Graal-At-Intel.pdf> by Jean-Philippe Halimi

[IonutBalosin.com/training](https://ionutbalosin.com/training)
 [@IonutBalosin](https://twitter.com/IonutBalosin)